

SpiceXpanse: A Scalable, Automated Framework for Efficient Parameter Optimization and Modeling of RRAM Circuits

Sai Sukruth Bezugam^{1,*}, Sanghyeon Choi¹, Stephan Menzel², Dmitri B. Strukov^{1,*}

¹Department of Electrical and Computer Engineering, University of California, Santa Barbara, CA, USA

²Forschungszentrum Jülich GmbH, Peter-Grünberg-Institut 7 (PGI-7), 52425 Jülich, Germany

*Corresponding authors: sbezugam@ece.ucsb.edu, strukov@ece.ucsb.edu

Abstract—The extraction and modeling of device parameters for resistive random-access memory (RRAM) circuits are hindered by significant device variability and high-dimensional parameter spaces. Traditional SPICE workflows depend on manual file manipulation, limiting both scalability and reproducibility. We introduce *SpiceXpanse*, an open-source framework that automates RRAM model calibration through configurable parameter exploration, parallel HSPICE execution, and interactive visualization. The modular optimization engine accommodates arbitrary sampling schemes, user-defined heuristics, and composite loss functions, yielding physically consistent fits while maintaining full procedural transparency. In preliminary tests on ten experimentally fabricated passive RRAM cells, the framework completed the entire extraction workflow without manual edits and leveraged parallel HSPICE runs to accelerate evaluation of candidate parameter sets. As a case study, we applied a shadow memory module—a series-resistance compensation technique that adds calibrated resistance values to homogenize device behavior—to align RESET thresholds at -2 V and significantly reduce threshold dispersion. *SpiceXpanse* thus offers a scalable, reproducible methodology that improves modeling accuracy and efficiency, with particular utility in neuromorphic and in-memory computing.

Index Terms—RRAM (Resistive Random Access Memory), SPICE-Based Optimization, Parameter Optimization, Memristor Modeling, Automated Calibration

I. INTRODUCTION

Memristive devices, particularly resistive random-access memory (RRAM) technologies, have advanced significantly; however, the design ecosystem continues to lack standardized, automated workflows for device-parameter extraction and compact-model fitting. Existing SPICE-based simulation and optimization workflows largely rely on labor-intensive file editing and offer limited automation, constraining the systematic exploration of high-dimensional parameter spaces [1], [2]. Early parameter-extraction efforts targeted specific RRAM models [3]–[5], but each relied on custom scripts with limited reusability. Recent tools have introduced automation,

such as the UNIMORE self-consistent extractor [9], a chaos-enhanced PSO fitter [10], a physics-informed RNN-based compact model [11], a derivative-free Bayesian optimizer for high-dimensional parameter sets [12], and Keysight’s commercial IC-CAP/Nowm suite that integrates hardware with proprietary extraction routines [13]. While these systems advance the state of the art, they are either model-specific, lack real-time visual feedback, or depend on proprietary toolchains—thereby limiting reproducibility and large-scale sweeps. As detailed in Table A1 (see Appendix A), existing tools lack the combination of open-source availability, parallel processing, and model-agnostic design that characterizes *SpiceXpanse*.

In addition, pronounced device-to-device and cycle-to-cycle variability necessitates exhaustive parameter sweeps, rendering manual or semi-automated approaches infeasible at scale [1], [2]. Recent studies have underscored the need for automated, scalable, and community-validated methodologies to support reproducible benchmarking and reliable circuit simulation, particularly for AI and neuromorphic accelerators that utilize large memristive crossbar arrays [1], [2].

In response, we present *SpiceXpanse*—an open-source framework that orchestrates parallel SPICE simulations, plug-and-play optimization heuristics, and composite multi-metric evaluation with real-time visual feedback, enabling reproducible and physically meaningful model fitting across heterogeneous memristive device arrays.

II. SPICEXPANSE

A. Framework Overview

SpiceXpanse automates the entire SPICE-based optimization workflow for RRAM devices. Figure 1a sketches the dataflow. The framework consumes three inputs: (i) user SPICE templates, (ii) experimental I–V data, and (iii) a YAML/JSON configuration that specifies sweep order, parameter bounds, sampling policy (linear, logarithmic, median, or explicit list), and metric weights. By adjusting, for ex-

Code available at <https://github.com/saibez/SpiceXpanse>

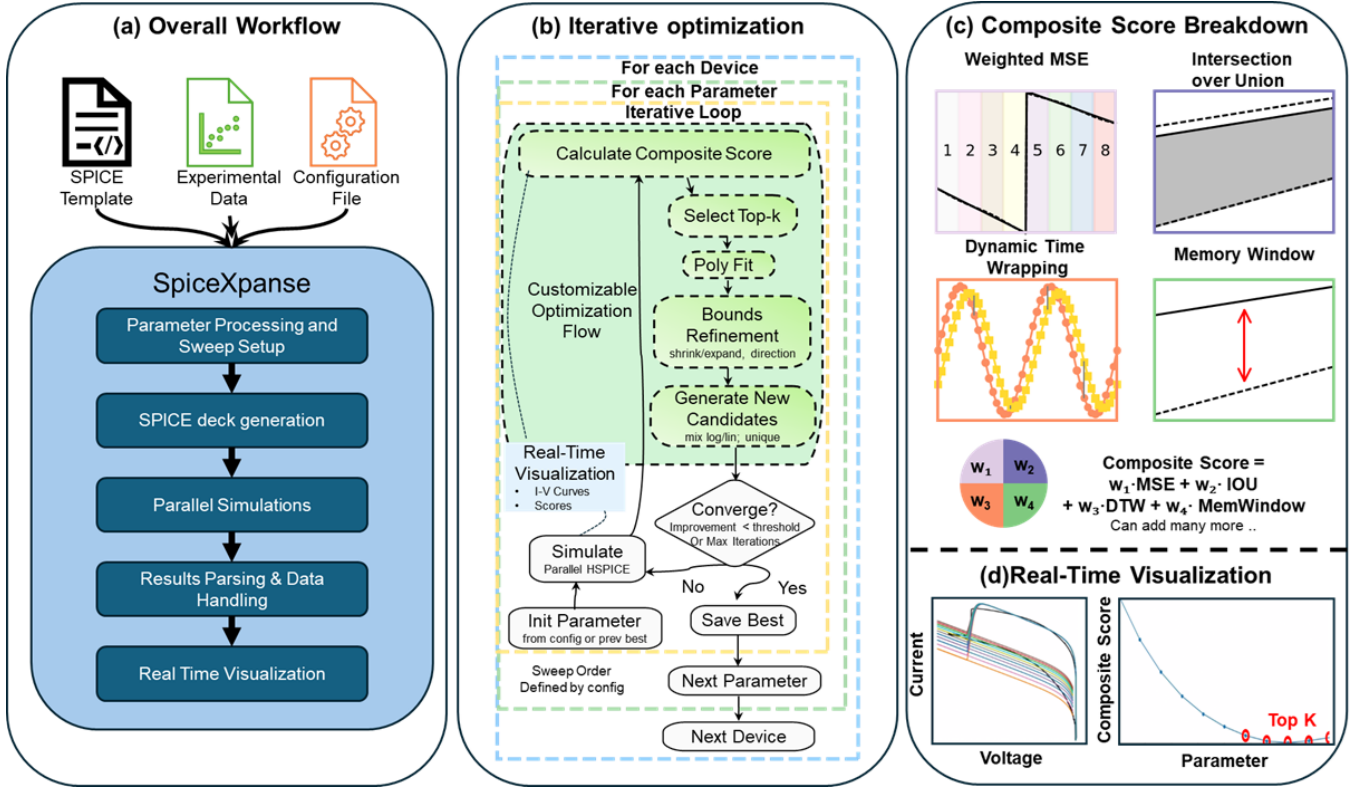


Fig. 1: Overview of *SpiceXpanse*—a scalable, automated framework for efficient parameter optimization and modeling of RRAM circuits. (a) Overall Workflow: *SpiceXpanse* integrates user-supplied SPICE templates, experimental I-V data, and configuration files to automate parameter processing, SPICE deck generation, parallel simulations, data parsing, and real-time visualization. (b) Iterative Optimization: For each device and parameter, the framework performs a configurable optimization loop, combining candidate selection, polynomial fitting, bounds refinement, and new candidate generation, with convergence based on composite score improvements. (c) Composite Score Breakdown: Composite scores are computed as weighted sums of user-defined metrics, such as weighted MSE, IOU, DTW, and memory window metrics, with flexibility to adjust or extend metrics and weights. (d) Real-Time Visualization: Simulated and experimental I-V curves, along with composite score trajectories, are updated at each iteration to provide transparent, interactive feedback during optimization.

ample, the Stanford-PKU parameter g_0 —which widens the HRS/LRS memory window in $I = I_0 \exp(-\text{gap}/g_0)$ —users can bias the fit toward memory-window fidelity.¹ After parsing the configuration, *SpiceXpanse* enumerates each sweep into an explicit candidate list, embeds candidates in SPICE netlists, and launches parallel HSPICE jobs (Fig. 1b). Raw `.tr0` results stream into Polars² data frames, are optionally compressed with `zstd`, and feed a pluggable optimization layer that scores candidates, refines bounds, and iterates until convergence (Section II-C). Simulated and experimental curves, as well as composite-score trajectories, are displayed live in an interactive dashboard (Fig. 1d). Upon completion the framework writes per-device parameter files and a full, reproducible log of all evaluations.

¹Sweep order and parameter coupling strongly influence convergence stability and physical realism in multi-parameter RRAM models [3], [4].

²A fast arrow/Rust DataFrame library

B. Iterative Optimization Engine

The iterative engine executes parameter sweeps device-by-device. Algorithm 1 gives pseudocode for a single parameter pass; Section II-C unpacks each step.

Parallel HSPICE back-end: Candidate netlists are dispatched to HSPICE in batch mode, co-ordinated by a lightweight thread pool. Raw `.tr0` traces stream into Polars data frames, where they can be (i) converted to Pickle for fast reuse and (ii) compressed with `zstd` when storage is a concern. Unless the user requests deep inspection, uncompressed traces are deleted after analysis to minimise disk pressure. All stages are profiled to balance CPU utilisation and I/O latency on commodity hardware.

Configurable inner loop: For each parameter the engine runs a fixed *outer* loop—advance device index, update bounds—while the *inner* optimization sequence is fully user-programmable. A YAML/JSON stanza or a GUI toggle (Fig. 1b, green block) can switch the inner routine from the default CANDIDATE → QUADRATIC-FIT → REFINE pipeline

to alternatives such as Bayesian optimization, stochastic sampling, or ML-guided heuristics.

Algorithm 1 Adaptive Parameter Optimization in SpiceXpanse

```

1: procedure ADAPTIVEOPT( $[l_0, u_0]$ ,  $p$ ,  $\mathcal{L}$ ,  $\lambda$ ,  $k$ ,  $I_{\max}$ )
2:    $i \leftarrow 0$ ,  $\mathcal{T} \leftarrow \emptyset$   $\triangleright$  evaluated points
3:    $\mathcal{C} \leftarrow \text{INITCANDIDATES}(l_0, u_0)$ 
4:   while  $i < I_{\max}$  do
5:     for all  $x \in \mathcal{C}$  do
6:       Run HSPICE on  $x$  and compute  $\mathcal{L}(x)$ 
7:        $\mathcal{B} \leftarrow \text{SELECTTOPK}(\mathcal{C}, k, \mathcal{L})$   $\triangleright$  exploit
8:       Fit  $f(x) = ax^2 + bx + c$  to  $\{(x, \mathcal{L}(x)) \mid x \in \mathcal{B}\}$ 
9:        $x^* \leftarrow \text{CLIP}(-\frac{b}{2a}, l_i, u_i)$ 
10:       $[l_{i+1}, u_{i+1}], \mathcal{R} \leftarrow \text{REFINE}(\mathcal{B}, [l_i, u_i], |\mathcal{T}|, i, \lambda)$ 
11:       $\mathcal{C} \leftarrow \{x^*\} \cup \mathcal{R}$   $\triangleright$  explore
12:       $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{C}$ ,  $i \leftarrow i + 1$ 
13:      if  $\text{CONVERGED}(\mathcal{T}, \mathcal{L})$  then
14:        return  $\text{ARGMIN}(\mathcal{T}, \mathcal{L})$ 

15: procedure REFINE( $\mathcal{B}$ ,  $[l, u]$ ,  $N_{\text{tot}}$ ,  $i$ ,  $\lambda$ )
16:    $m \leftarrow \min \mathcal{B}$ ;  $n \leftarrow \max \mathcal{B}$ ;  $\delta \leftarrow \frac{n-m}{2}$ 
17:    $[l', u'] \leftarrow [\max(l, m - \delta), n + \delta]$ 
18:   if  $(m - l) < 0.05(u - l) \vee (u - n) < 0.05(u - l)$ 
19:     then
20:       expand  $[l', u']$  by  $\text{expansion\_factor} \times (n - m)$ 
21:    $l' \leftarrow \max(l', 10^{-12})$ ;
22:   if  $l' > u'$  then swap  $l', u'$ 
23:    $r \leftarrow u'/l'$ ;  $\phi \leftarrow \begin{cases} 0.8, & r > 100 \\ 0.5, & 10 < r \leq 100 \\ 0.2, & r \leq 10 \end{cases}$ 
24:    $N \leftarrow \max(5, \lfloor \frac{N_{\text{tot}}}{2} \lambda^i \rfloor)$ 
25:   Draw  $N$  samples:  $\lceil \phi N \rceil$  logarithmic,  $\lfloor (1 - \phi)N \rfloor$  linear; reject repeats
26:   return  $[l', u'], \mathcal{R}$ 

```

C. Adaptive Optimization Routine (Implementation Detail)

Algorithm 1 and Table I outline the adaptive exploration–exploitation loop executed for *each* model parameter. The procedure balances fast convergence with safeguards against local minima as follows:

- 1) **Initialisation** – INITCANDIDATES draws an evenly or log-spaced set $\mathcal{C}$ within the user bounds $[l_0, u_0]$. The global set $\mathcal{T}$ stores every point evaluated so far.
- 2) **Evaluation** – Each $x \in \mathcal{C}$ is embedded in a SPICE netlist, simulated with HSPICE, and scored by the composite loss $\mathcal{L}(x)$
- 3) **Exploitation** – SELECTTOPK keeps the k lowest-loss candidates $\mathcal{B}$. A quadratic surrogate $f(x) = ax^2 + bx + c$ is fitted to $(x, \mathcal{L}(x))$ pairs in $\mathcal{B}$, and its vertex $x^* = -b/(2a)$ is clipped to $[l_i, u_i]$.
- 4) **Adaptive refinement** – REFINE recentres the window on $\{\min \mathcal{B}, \max \mathcal{B}\}$, re-expands if the best point sits

TABLE I: Notation used in Algorithm 1

Symbol	Meaning
$[l_i, u_i]$	Current lower/upper bounds at epoch i
p	Index of the parameter being tuned
$\mathcal{L}$	Composite loss function (to be minimised)
λ	Bound-shrink decay factor per epoch
k	Top- k candidates retained for exploitation
$I_{\max}$	Maximum optimization epochs
$\mathcal{C}$	Candidate set evaluated at current epoch
$\mathcal{B}$	Best k candidates ($ \mathcal{B} = k$)
$\mathcal{T}$	Union of all evaluated points
x^*	Parabola vertex after clipping to $[l_i, u_i]$
r	Current span ratio, u'/l'
ϕ	Fraction of logarithmic samples in $\mathcal{R}$
N	Number of new samples generated in REFINE

within 5 % of a boundary, enforces a hard floor of 10^{-12} for logarithmic sampling, and chooses the log-vs-linear split ϕ via the span ratio $r = u'/l'$: $\phi = 0.8, 0.5, 0.2$ for wide/medium/narrow windows, respectively.

- 5) **Exploration** – The next candidate set $\mathcal{C} = \{x^*\} \cup \mathcal{R}$ mixes one *exploitative* point x^* with $N = \max(5, \lfloor |\mathcal{T}|/2 \lambda^i \rfloor)$ *exploratory* samples, where the decay factor $0 < \lambda < 1$ shrinks N geometrically as the search progresses.
- 6) **Convergence** – Terminate when (i) absolute and relative loss improvements fall below user thresholds, (ii) the window width $u_i - l_i$ drops under tolerance, or (iii) the epoch cap $I_{\max}$ is reached. The routine returns $\text{argmin}_{x \in \mathcal{T}} \mathcal{L}(x)$.

Parallelism and complexity: All candidates in $\mathcal{C}$ are simulated on separate threads for a *single* device, while devices are processed sequentially. The worst-case cost per parameter is therefore $O(k + \sum_{i=0}^{I_{\max}} N_i)$ HSPICE runs, with $N_i \propto \lambda^i$.

D. Composite Score and Metrics

SpiceXpanse builds a composite loss $\mathcal{L} = \sum_j w_j m_j$ at run-time, where the metric–weight pairs $\{(m_j, w_j)\}$ are fully user-defined and normalised ($\sum_j w_j = 1$). Built-in metrics include (i) region-weighted MSE for voltage-specific fidelity, (ii) intersection-over-union (IoU) to measure curve overlap, (iii) dynamic time warping (DTW) for shape alignment, and (iv) memory-window scores that target Stanford–PKU behaviour (Fig. 1(c)).

Users can introduce additional metrics—e.g., a SET-overshoot penalty or RESET-slope uniformity—via a one-line Python callback or a YAML block. Weights may be adjusted interactively in the GUI or pre-set in the configuration file; any change is applied at the next epoch so the search direction adapts in real time. A second weighting layer can be enabled to compute separate composite scores for SET and RESET traces before merging them, giving fine-grained control over competing objectives. All per-metric traces are logged and plotted beside the total loss for easy diagnosis of optimization progress.

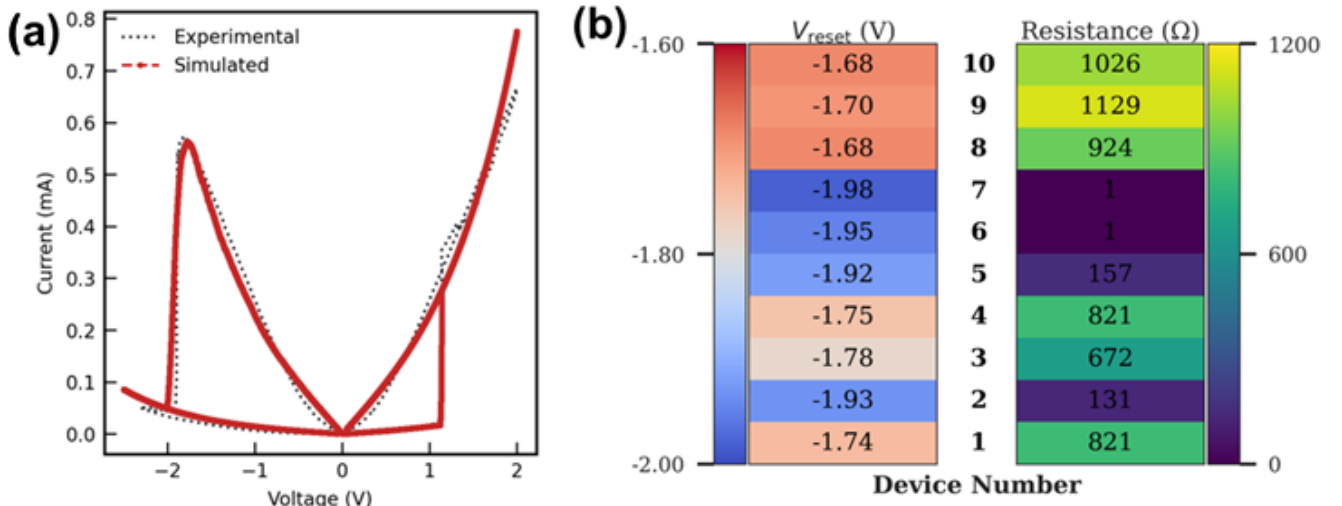


Fig. 2: (a) Overlay of experimental and simulated I–V curves for a representative passive RRAM device, demonstrating the high-quality fit achieved by automated parameter extraction in *SpiceXpanse*. (b) Shadow-memory compensation: fitted RESET threshold voltage for each device (left) and the optimal series resistance (right) required to shift the RESET threshold to a uniform -2 V target, reducing threshold dispersion across the array.

E. Real-Time Visualisation Dashboard

SpiceXpanse ships with a lightweight GUI that streams results as the optimizer runs. After every HSPICE batch the dashboard

- overlays simulated and experimental I–V curves, updating the plot in ≤ 100 ms;
- charts the composite-score trajectory versus the swept-parameter value (Fig. 1(d));
- highlights the current top- k candidate set for full transparency.

Interactive controls let the user toggle log/linear scaling, zoom, and adaptive down-sampling so large data sets remain responsive. All artefacts—optimised parameter files, per-epoch score logs, and PNG snapshots of the plots—are written to a time-stamped results folder. The dashboard state itself is serialized to a `.json` session file, enabling exact experiment replay or further off-line analysis.

III. CASE STUDY: AUTOMATED FITTING AND SHADOW MEMORY ENGINEERING IN RRAM ARRAYS

To benchmark *SpiceXpanse* under realistic laboratory conditions, experimental I–V data from ten passive RRAM devices fabricated following the protocol of Kim *et al.* [8] were automatically fitted to the Stanford–PKU Verilog-A model [4] using the composite-score workflow (Fig. 1b). Parameter-specific weights (g_0 : memory-window fidelity, β : RESET-curvature exponent) and iterative settings (top_k=7, decay_factor=0.5, expansion_factor=0.8, max_iterations=10) yielded high-quality overlays between simulation and measurement for all ten devices (Fig. 2a shows a representative fit for one device).

The fitted RESET threshold voltages exhibited pronounced device-to-device variability (Fig. 2b, left). Applying the shadow-memory concept of Bezugam *et al.* [7], *SpiceXpanse* computed the series resistance, R_{series} , required to shift each device’s RESET threshold to a uniform -2 V target. The resulting optimal resistance values (Fig. 2b, right) significantly reduced threshold dispersion, demonstrating the framework’s ability to guide circuit-level mitigation of intrinsic variability.

This study underscores both the scalability of *SpiceXpanse* for batch device modeling and its utility in supporting adaptive hardware techniques that homogenize switching behavior in memristive arrays.

IV. DISCUSSION

While *SpiceXpanse* significantly reduces manual effort and improves reproducibility in device modeling, several limitations remain. First, the accuracy of extracted parameters and the effectiveness of threshold engineering are inherently bounded by the underlying compact model’s ability to capture all relevant physical mechanisms and variability sources. Second, although the batch-optimization workflow is scalable, the computational cost can become substantial for very large device arrays or high-fidelity models, particularly given the reliance on repeated HSPICE runs. Migrating the back-end to an open-source, parallel-capable engine—*ngspice* with OpenMP threading [14] or *Xyce* with MPI scaling [15]—could unlock greater multi-core. Third, the current framework has been primarily validated on passive RRAM devices and the Stanford–PKU model; its generalizability to other device types, more advanced array architectures, or models incorporating additional non-idealities (e.g., stochastic switching, temperature effects, or selector

integration) requires further study. Looking ahead, several directions could further enhance the impact and utility of *SpiceXpanse*. Integrating machine-learning-based surrogate models may accelerate parameter optimization and enable real-time experiment replication. Extending support to a broader range of memristive and emerging memory devices, as well as mixed-signal and neuromorphic circuits, would increase the platform’s versatility. Finally, tighter integration with experimental workflows—including automated pre-simulation checks and direct feedback to device fabrication—could effectively close the loop between modeling and hardware development, enabling adaptive and robust design for next-generation memory and computing systems. By openly releasing *SpiceXpanse* to the community, we aim to foster reproducible benchmarking and collaborative progress in RRAM modeling, circuit optimization, and adaptive hardware research.

APPENDIX A

TABLE A1: Comparison of Automated RRAM Parameter-Extraction Tools

Tool*	Open-src	Par. SPICE	Model-agn.	Optimiser†
UNIMORE-22 [9]	× ¹	—	× ³	NM
Chaos-PSO-24 [10]	× ¹	—	✓	CPSO
PI-RNN-23 [11]	× ¹	N/A	✓	PRNN
Keysight IC-CAP [13]	× ²	Opt.	×	LSQ
SpiceXpanse (this work)	✓ ⁴	✓ ⁶	✓ ⁵	APF ⁷

† APF = adaptive poly-fit (default). Other codes: NM = Nelder–Mead, CPSO = chaotic PSO, PRNN = physics-informed RNN, LSQ = least-squares.

¹ Provides no public repository. ² Proprietary. ³ Targets a UNIMORE-specific Verilog-A model. ⁴ MIT-licensed; full source on GitHub. ⁵ Plugin architecture supports any Verilog-A / SPICE compact model via text-template substitution and custom metrics. ⁶ Threaded HSPIICE back-end; ngspice/Xyce adapters planned. ⁷ Any optimiser (PSO, Bayesian, GA, etc.) can be registered via `register_optimizer()`.

ACKNOWLEDGMENT

This work was supported by NSF EFRI Grant #2318152 and ONR grant #N00014-22-1-2842.

REFERENCES

- [1] M. Lanza *et al.*, “The growing memristor industry,” *Nature*, vol. 640, no. 8059, pp. 613–622, Apr. 2025, doi:10.1038/s41586-025-08733-5.
- [2] F. Qin, Y. Zhang, H. W. Song, and S. Lee, “Enhancing memristor fundamentals through instrumental characterization and understanding reliability issues,” *Mater. Adv.*, vol. 4, pp. 1850–1875, 2023, doi:10.1039/D3MA00069A.
- [3] J. Reuben, D. Fey, and C. Wenger, “A modeling methodology for resistive RAM based on the Stanford–PKU model with extended multilevel capability,” *IEEE Trans. Nanotechnol.*, vol. 18, pp. 647–656, Jun. 2019, doi:10.1109/TNANO.2019.2922838.
- [4] Z. Jiang *et al.*, “A compact model for metal–oxide resistive random access memory with experimental verification,” *IEEE Trans. Electron Devices*, vol. 63, no. 5, pp. 1884–1892, May 2016, doi:10.1109/TED.2016.2545412.
- [5] D. Maldonado-Correa *et al.*, “Parameter extraction techniques for the analysis and modeling of resistive memories,” *Microelectron. Eng.*, vol. 265, Art. no. 111876, Sep. 2022, doi:10.1016/j.mee.2022.111876.
- [6] R. Vink, “*pola-rs/Polars: Python Polars 0.16.11*,” Zenodo, 2023, doi:10.5281/zenodo.7699984.
- [7] S. S. Bezugam *et al.*, “Controlling ReRAM’s switching characteristics with shadow memory for continual learning,” in *Proc. 17th IEEE Int. Memory Workshop (IMW)*, Monterey, CA, USA, May 2025, pp. 1–4, doi:10.1109/IMW61990.2025.11026884.
- [8] H. Kim *et al.*, “4 K-memristor analog-grade passive crossbar circuit,” *Nat. Commun.*, vol. 12, Art. no. 5198, Aug. 2021, doi:10.1038/s41467-021-25455-0.
- [9] T. Zanotti, P. Pavan, and F. M. Puglisi, “Self-consistent automated parameter extraction of RRAM physics-based compact model,” in *Proc. ESSDERC*, 2022, pp. 316–319, doi:10.1109/ESSDERC55479.2022.9947161.
- [10] L. Wang and Y. Wu, “Parameter extraction of memristor model based on improved PSO with chaotic inertia,” in *Proc. PEAI*, 2024, pp. 515–518, doi:10.1145/3674225.3674318.
- [11] Y. Sha, J. Lan, Y. Li, and Q. Chen, “A physics-informed recurrent neural network for RRAM modeling,” *Electronics*, vol. 12, no. 13, Art. no. 2906, 2023, doi:10.3390/electronics12132906.
- [12] R. P. Martínez *et al.*, “Compact model parameter extraction via derivative-free optimization,” *IEEE Access*, vol. 12, pp. 123224–123235, 2024, doi:10.1109/ACCESS.2024.3453198.
- [13] A. Nugent and K. Campbell, “Knowm/PathWave IC-CAP memristor modeling suite,” 2023. [Online]. Available: <https://knowm.org/knowm-memristor-modeling-and-simulation/>.
- [14] H. Vogt, G. Atkinson, and P. Nenzi, *ngspice User’s Manual*, Version 42, Dec. 2023. [Online]. Available: <https://ngspice.sourceforge.io/docs/ngspice-42-manual.pdf>.
- [15] E. R. Keiter *et al.*, *Xyce Parallel Electronic Simulator Reference Guide*, Version 7.9, Sandia Natl. Labs., Oct. 2024. [Online]. Available: <https://xyce.sandia.gov/>.